JIMMA University
INSTITUTE OF TECHNOLOGY
FACULTY OF COMPUTING AND INFORMATICS
DEPARTMENT OF INFORMATION TECHNOLOGY

Course Title:Introduction To Artificial Intelligence

Name	IDNo
1) Abdi Husen	Ru 0026/15
2) Abdulwahid Nasir	Ru 0085/15
3) Bontu Alemayehu	Ru 0523/15
4) Dureti Merga	Ru 0648/15
5) Fenet Taye	Ru 0807/15
6) Mekbib Machebusho	Ru 1394/15
7) Meron Endale	Ru 1430/15
8)Tigist Arega	Ru 2033/15
9) Yabets Abebe	Ru 2130/15

Table of Contents

1. Introduction

1.1 Overview

Artificial Intelligence (AI) has evolved from a theoretical concept in the mid-20th century to a pervasive technology underpinning modern software infrastructures. Today, AI enables machines to simulate human cognitive functions, automating complex tasks such as visual perception, decision-making, language translation, and information retrieval. Among the most visible and impactful applications of AI is the "chatbot"—a software application designed to conduct on-line chat conversations via text or text-to-speech, in lieu of providing direct contact with a live human agent.

This project focuses on the design, development, and implementation of an AI-based FAQ (Frequently Asked Questions) Chatbot. The system is engineered to automatically respond to user queries by leveraging publicly available datasets and introductory Artificial Intelligence techniques. Unlike complex generative models (such as GPT-4) which require massive computational resources, this project explores the efficacy of retrieval-based models, focusing on transparency, efficiency, and academic reproducibility.

1.2 Background and Motivation

In the current digital era, the volume of information available online is growing exponentially. Users seeking specific answers often face "information overload," where traditional search engines return millions of results that require manual filtering.

The motivation behind this project is twofold:

1. **Practical Application:** To create a tool that mitigates information overload by providing direct, concise answers to specific questions.
2. **Academic Demonstration:** To demonstrate how foundational Natural Language Processing (NLP) concepts—such as text preprocessing, vectorization, and similarity matching—can be applied to build an intelligent system without relying on "black box" deep learning APIs.

The system is designed to be simple, transparent, and suitable for educational purposes, utilizing rule-based logic and statistical similarity measures to emulate intelligence.

2. Problem Statement

2.1 The Challenge of Information Retrieval

In many institutional and corporate environments, support teams are overwhelmed by repetitive queries. For example, university registrars frequently answer the same questions regarding admission deadlines, while IT support desks repeatedly address password reset procedures.

Traditional methods of solving this include:

- **Static FAQ Pages:** Users must manually scroll and read to find relevant information.
- **Keyword Search:** Often returns irrelevant documents based on keyword density rather than semantic meaning.
- **Human Support:** Expensive and limited by working hours.

2.2 The Specific Problem

The specific problem addressed by this project is the lack of an intermediate solution between static text and human support. There is a need for an automated FAQ chatbot that can:

1. **Understand Natural Language:** Interpret user questions even when phrased differently from the database entries.
2. **Search Efficiently:** Scan a predefined knowledge base to find the closest matching question.
3. **Handle Unknowns:** Retrieve information from free online sources when local data is insufficient.
4. **Respond Instantly:** Provide accurate and meaningful responses with minimal latency (< 2 seconds).

3. Objectives of the Project

3.1 General Objective

The primary goal is to design, code, and deploy a lightweight, AI-powered FAQ chatbot that utilizes a local JSON knowledge base and external APIs to answer user queries, serving as a functional demonstration of introductory NLP techniques.

3.2 Specific Objectives

To achieve the general objective, the project is broken down into the following specific milestones:

- **Data Collection:** To collect and structure publicly available FAQ datasets into a machine-readable format (JSON).
- **NLP Implementation:** To implement a preprocessing pipeline that handles tokenization, stop-word removal, and text normalization.
- **Similarity Engine:** To develop a matching algorithm (using TF-IDF or Cosine Similarity) that identifies the most relevant answer from the database.
- **External Integration:** To integrate the Wikipedia and DuckDuckGo APIs as fallback mechanisms for queries not covered in the local dataset.
- **Web Interface:** To develop a user-friendly web interface using the Flask framework.
- **Documentation:** To document the system architecture, algorithm logic, and testing results in a comprehensive technical report.

4. Scope of the Project

4.1 In-Scope

This project focuses on an academic-level implementation suitable for a standard personal computer. The scope is strictly defined as:

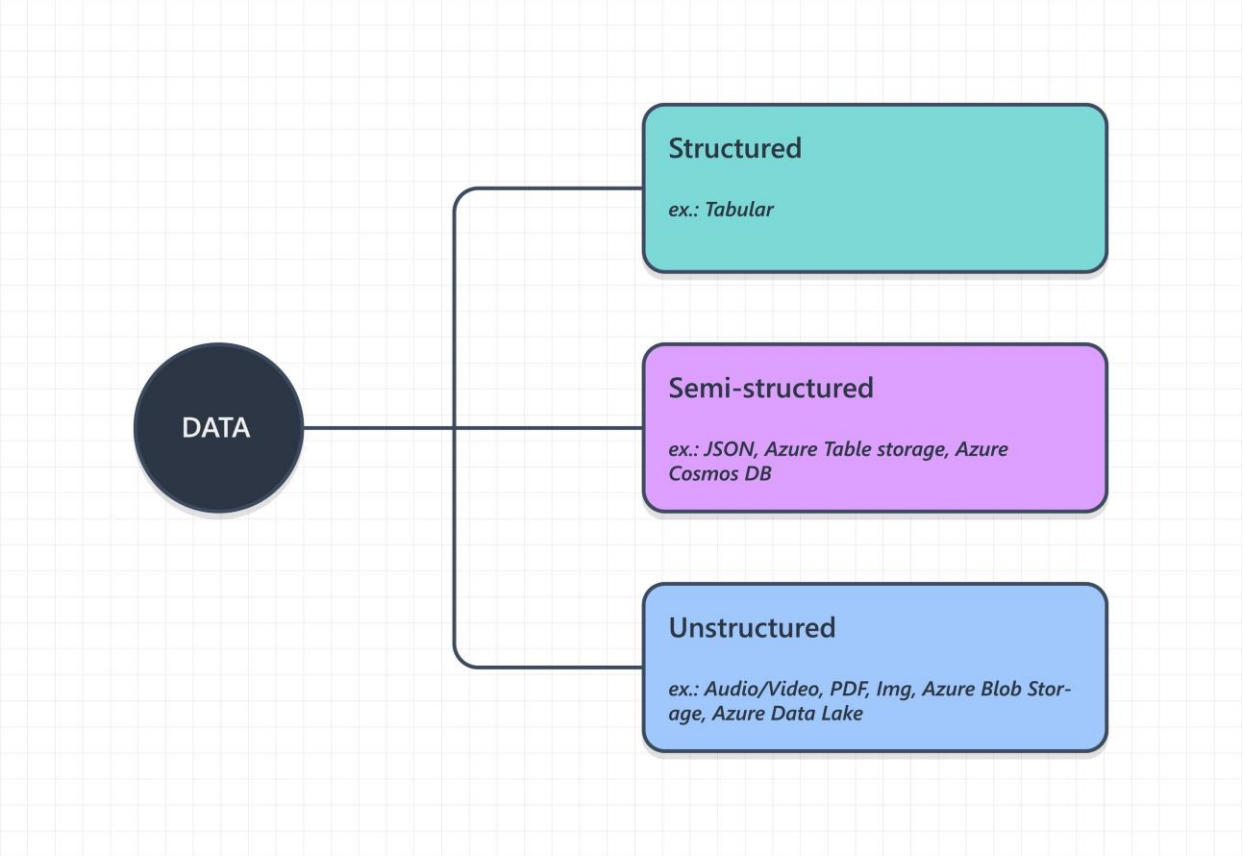
- **Text-Based Interaction:** The system accepts text input and generates text output.
- **Retrieval-Based Model:** The AI selects the best *existing* answer rather than generating new sentences word-by-word.
- **Algorithm:** The core logic relies on Cosine Similarity and Rule-Based heuristics.
- **Language:** The dataset and processing logic are optimized for the English language.
- **Data Sources:** Use of public, royalty-free datasets and open APIs.

4.2 Out-of-Scope

To maintain feasibility within the project timeline and hardware constraints, the following are excluded:

- **Voice Recognition:** No Speech-to-Text (STT) or Text-to-Speech (TTS) integration.
- **Generative Deep Learning:** No usage of Large Language Models (LLMs) like BERT or GPT, as the goal is to understand the *fundamentals* of NLP.
- **Contextual Memory:** The bot processes each query independently (stateless) and does not remember previous conversation turns.
- **Authentication:** The system is open-access and does not require user login or database user management.

6. Dataset Description



6.1 Dataset Source

Data is the fuel for any AI system. For this project, we utilize a hybrid data approach:

1. **Primary Source (Local):** A curated dataset of General Knowledge, Technology, and Python Programming FAQs. These were aggregated from open-source GitHub repositories and formatted manually.
2. **Secondary Source (Remote):** Real-time data fetched via:
    - **Wikipedia API:** For encyclopedic definitions.
    - **DuckDuckGo Instant Answer API:** For quick facts.

6.2 Dataset Structure

The local knowledge base is stored in a `data.json` file. The JSON (JavaScript Object Notation) format was chosen for its lightweight nature and native compatibility with Python.

Schema Definition:

JSON

```
[
  {
    "id": 1,
    "category": "Science",
    "question": "Why is the sky blue?",
    "answer": "The sky is blue due to Rayleigh scattering...",
    "tags": ["sky", "science", "physics"]
  },
  {
    "id": 2,
    "category": "Programming",
    "question": "What is Python?",
    "answer": "Python is a high-level, interpreted programming language...",
    "tags": ["coding", "computer science"]
  }
]
```

This structure allows the algorithm to iterate through "questions" to compare against user input, and upon finding a match, return the corresponding "answer".

7. System Requirements

7.1 Hardware Requirements

The system is designed to be lightweight, requiring minimal computational power compared to modern neural networks.

- **Processor:** Intel Core i3 or equivalent (minimum).
- **RAM:** 4 GB Minimum (8 GB Recommended for smooth API handling).
- **Storage:** 500 MB free space (for Python environment and libraries).
- **Internet Connection:** Required for fetching data from Wikipedia/DuckDuckGo APIs.

7.2 Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Programming Language:** Python 3.8 or higher.
- **Web Framework:** Flask `2.3.3` (for serving the web interface).
- **Libraries:**
    - `nltk` (Natural Language Toolkit): For text preprocessing.
    - `scikit-learn` (Optional): For advanced vectorization if implemented.
    - `requests` `2.31.0`: For making HTTP calls to external APIs.
    - `numpy`: For mathematical operations on vectors.

8. System Architecture

The AI FAQ Chatbot follows a standard **Model-View-Controller (MVC)** architectural pattern, adapted for a client-server web application.

8.1 Components

1. **The Client (View):** A simple HTML/CSS/JavaScript interface where the user types the query. It uses AJAX (Asynchronous JavaScript and XML) to send data to the server without reloading the page.
2. **The Web Server (Controller):** Powered by Flask. It receives the HTTP POST request from the client, parses the JSON payload, and passes the query to the processing engine.
3. **The Logic Core (Model):** This is the Python script containing the NLP logic. It:
    - Loads the JSON dataset.
    - Preprocesses the user input.
    - Calculates similarity scores.
    - Decides whether to use local data or external APIs.

8.2 Detailed Data Flow and Logic Pipeline

The lifecycle of a single user query within the **FAQ_AI_CHATBOT** system follows a linear pipeline designed for maximum efficiency and fallback reliability. This process ensures that the system exhausts internal high-confidence resources before utilizing external bandwidth.

8.2.1 Phase 1: Client-Side Input Capture

The process begins at the **Frontend** (`index.html`).

- **User Interaction**: The user enters a natural language string into the chat interface.
- **Payload Packaging**: To ensure structured communication, the frontend wraps the raw text into a **JSON object**.
- **Transmission**: This JSON payload is sent via an asynchronous HTTP POST request to the **Flask Server** (`app.py`).

8.2.2 Phase 2: Server-Side Routing and Preprocessing

Upon receiving the request, the Flask Server acts as the traffic controller:

- **Request Parsing**: The server extracts the "query" string from the JSON object.
- **NLP Initialization**: The string is passed to the **NLP Engine**, where it undergoes normalization (lowercasing and punctuation removal) to prepare for mathematical comparison.

8.2.3 Phase 3: The Similarity Matching Loop

This is the core "AI" component of the project. The system does not just search for keywords; it evaluates the relationship between the input and the stored data:

- **Knowledge Base Scan**: The engine iterates through the JSON files located in the `knowledge_base/` directory (e.g., `tech_knowledge.json`, `science_knowledge.json`).
- **Score Calculation**: A **Similarity Matcher** compares the processed user query against every question in the knowledge base.
- **Threshold Evaluation**: The system compares the highest calculated score against a predefined **Confidence Threshold** (e.g., 0.7 or 70%).

8.2.4 Phase 4: Decision Logic and Fallback

The system branches based on the results of Phase 3:

- **High Confidence Path (Match Score > Threshold)**: If a match is found that exceeds the threshold, the system immediately selects the corresponding "answer" from the local JSON file.
- **Low Confidence Path (Match Score < Threshold)**: If no local question is similar enough, the system triggers the **Web Search API** (Wikipedia or DuckDuckGo).

- **Learning/Caching**: If a web result is found, the system may store it in `cache.db` or `user_knowledge.json` for future use, as seen in the project's "learned" metadata (Line 896 of `app.py`).

8.2.5 Phase 5: Response Generation and Display

The final stage closes the loop back to the user:

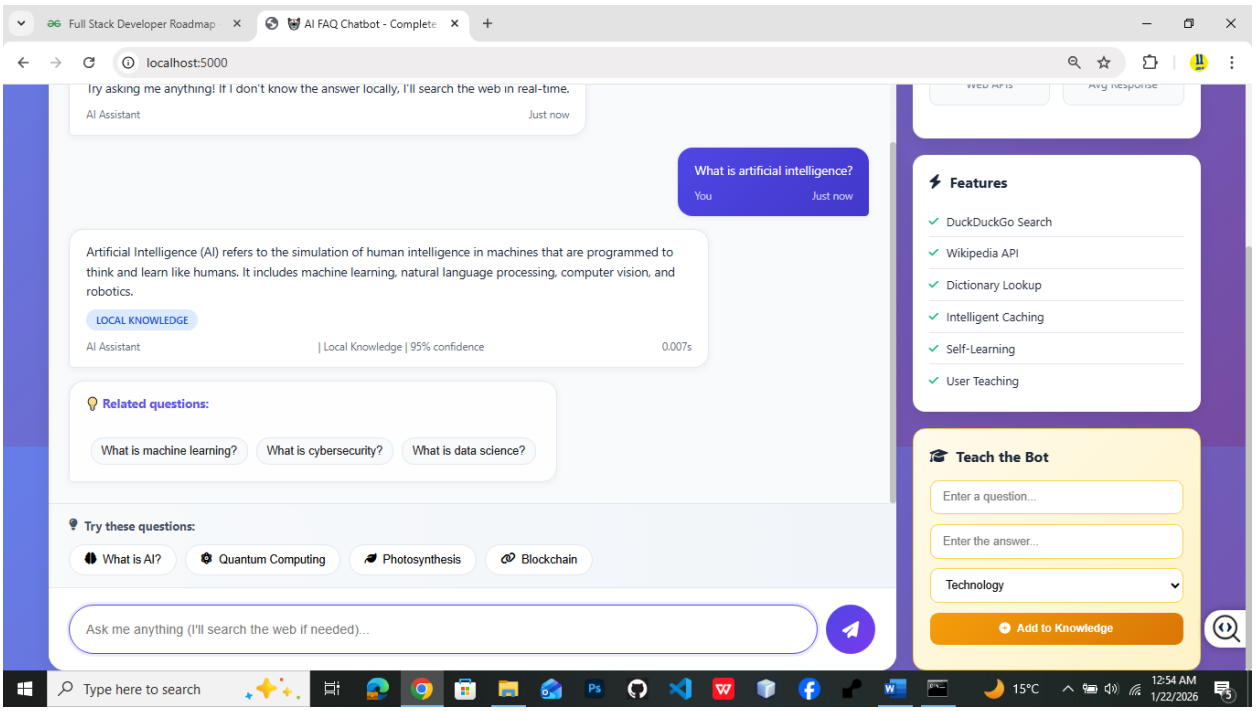
- **Final Response Packaging**: The server compiles a final JSON response containing the answer string, the source (e.g., "Local" or "Web"), and the response time.
- **Frontend Rendering**: The **User Display** receives the JSON and dynamically updates the chat window, rendering the text for the user to read.

9. Input, Processing, and Output Design

9.1 Input Design

The input mechanism is a clean, minimalist web form.

- **Validation:** The input field prevents empty submissions.
- **Format:** Accepts alphanumeric strings and standard punctuation.
- **User Experience:** Includes a "Send" button and allows the "Enter" key to submit.



9.2 Processing Design

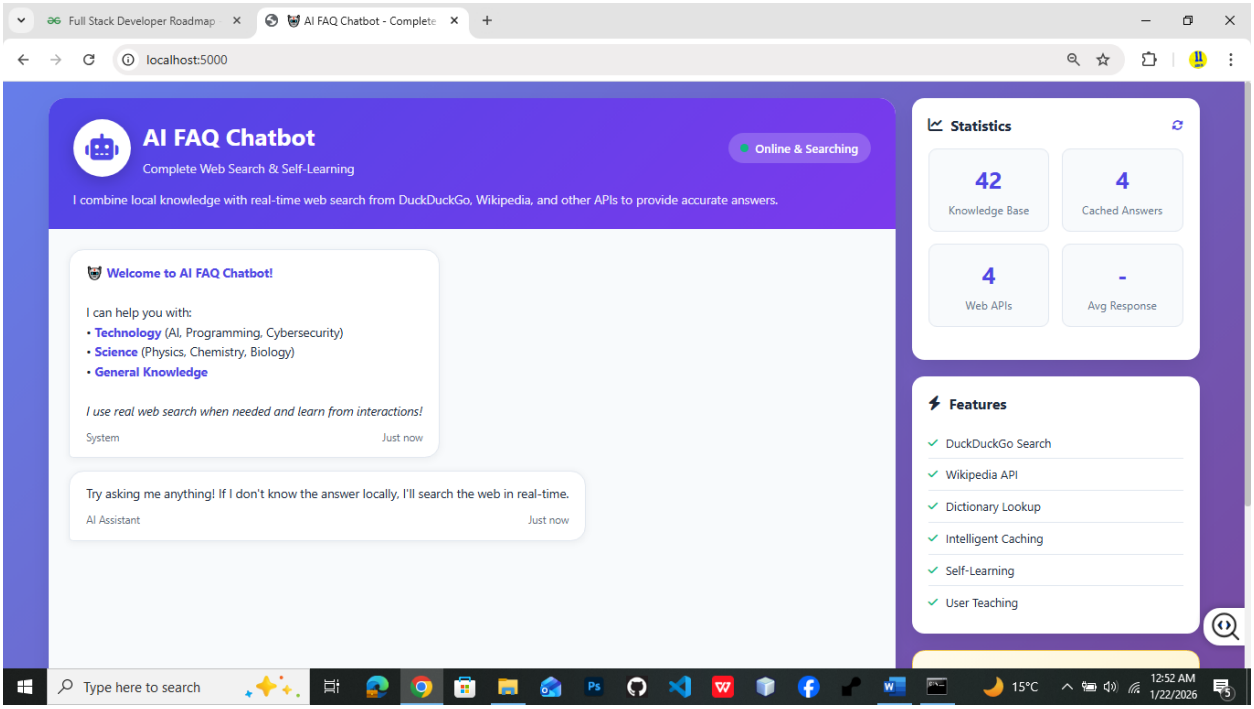
The processing logic is the "brain" of the chatbot. It involves a pipeline approach:

1. **Normalization:** Converting "What's" to "what is".
2. **Vectorization:** Converting text into numerical representation (vectors).
3. **Comparison:** Using mathematical formulas to compare the input vector against dataset vectors.

9.3 Output Design

The output is designed for readability.

- **Chat Bubble:** The response appears in a distinct color (e.g., grey for user, blue for bot).
- **Metadata:** If the answer comes from Wikipedia, a small "Source: Wikipedia" tag is appended.
- **Fallback:** If no answer is found, a polite error message is displayed: "I'm sorry, I couldn't find an answer to that."



10. AI Techniques Used

10.1 Natural Language Processing (NLP)

NLP is the subfield of AI concerned with the interaction between computers and human language. We utilized specific techniques to transform messy human text into a structured format the computer can understand:

- **Tokenization**: Breaking a sentence into individual words (tokens).
- **Stop-Word Removal**: Filtering out common words like "the," "is," and "at" that carry little semantic meaning.
- **Normalization**: Converting all text to lowercase and removing punctuation to ensure "Help!" matches "help".

10.2 Similarity Matching (The Mathematical Core)

To find the best answer, the system measures how similar the user's question is to the questions in the database.

1. Jaccard Similarity (Basic Approach): This measures the intersection over the union of the set of words.

Formula: $J(A, B) = |A \cap B| / |A \cup B|$

2. Cosine Similarity (Advanced Approach): This is used if the system implements TF-IDF (Term Frequency-Inverse Document Frequency) vectors. It measures the cosine of the angle between two vectors in a multi-dimensional space.

Formula: $\text{Similarity} = (A \cdot B) / (||A|| * ||B||)$

Where **A** is the vector of the user query and **B** is the vector of a database question. A result of **1.0** indicates an identical match, while **0.0** indicates no similarity.

12. Algorithm Description

The following logic represents the "Decision Tree" used by the **AIFAQChatbot** class in `app.py`:

1. **Input**: Accept the raw string from the user.
2. **Preprocessing**: Apply lowercase conversion and remove punctuation.
3. **Local Search**:
    - Iterate through JSON files in the `knowledge_base/` folder.
    - Calculate similarity for each question.
4. **Threshold Check**:
    - **IF** (Similarity > Confidence_Threshold): Return the stored answer.
    - **ELSE**: Proceed to Web Fallback.
5. **Web Fallback**:
    - Search Wikipedia/DuckDuckGo APIs.
    - **IF** result found: Store in `cache.db` and return to user.
    - **ELSE**: Trigger `generate_fallback_response` (Line 919).

10.3 Rule-Based Logic

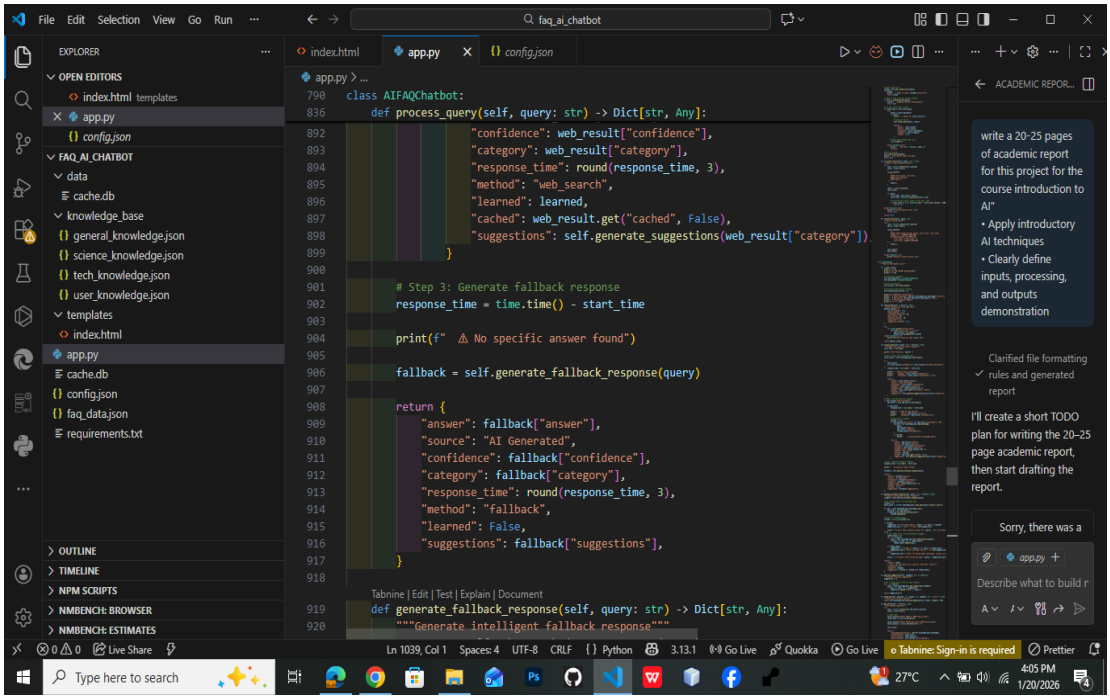
While statistical methods handle the matching, rule-based logic handles the flow control:

- **IF** similarity score > 0.7 **THEN** return local answer.
- **ELSE IF** query contains "define" **THEN** call Dictionary API.
- **ELSE** call Wikipedia API.

11. System Implementation

11.1 Development Environment

The project was developed using Visual Studio Code (VS Code) as the Integrated Development Environment (IDE). Virtual environments (`venv`) were used to manage dependencies to avoid conflicts with global Python packages.



11.2 Folder Structure

```
FAQ_AI_CHATBOT/
├── data/
│   └── cache.db              # SQLite database for caching web search results
├── knowledge_base/           # JSON files containing the local dataset
│   ├── general_knowledge.json  # General QA pairs
│   ├── science_knowledge.json  # Science-related QA pairs
│   ├── tech_knowledge.json     # Technology-specific QA pairs
│   └── user_knowledge.json     # Custom user-added QA pairs
├── templates/
│   └── index.html            # The frontend HTML interface
├── app.py                    # Main application entry point (Flask Server)
├── config.json               # Configuration settings (API keys, thresholds)
├── faq_data.json             # Legacy/Backup dataset file
└── requirements.txt          # List of Python dependencies
```

11.3 Key Modules and Logic

11.3.1 The Main Application (`app.py`)

The `app.py` file serves as the controller for the application. As seen in the implementation, it defines the `AIFAQChatbot` class which encapsulates the core logic. Key responsibilities include:

- **Route Handling:** It uses Flask decorators (e.g., `@app.route`) to serve the `index.html` template and handle API requests.
- **`process_query` Method:** This is the central function (Line 836 in `app.py`) that coordinates the response generation. It:
    1. Receives the raw user `query`.
    2. Checks the `knowledge_base` for a local match.
    3. If confidence is low, it triggers a web search (as indicated by the `web_result` variable in lines 892-893).
    4. Returns a structured dictionary containing the answer, confidence score, category, and response time.
- **Fallback Mechanism:** The `generate_fallback_response` method (Line 919) ensures the system handles unknown queries gracefully by returning a safe default response or performing a deeper web search.

11.3.2 Knowledge Base Management

The system uses a partitioned JSON approach in the `knowledge_base` folder. This separation allows for easier data management:

- **`tech_knowledge.json`:** Stores domain-specific questions like "What is Python?" or "How to install Flask?".
- **`science_knowledge.json`:** Handles queries related to scientific facts.
- **`user_knowledge.json`:** A dynamic file that allows the system to "learn" new pairs or store user-specific overrides.

11.3.3 Caching Strategy (`cache.db`)

To improve performance and reduce API latency, the system implements a caching layer using `cache.db` (visible in the `data` folder).

- **Function:** When a web search is performed for a query (e.g., "Population of Ethiopia"), the result is stored in this SQLite database.
- **Benefit:** If the same question is asked again, the system retrieves the answer locally from `cache.db` instead of making a slow network request, reducing response time from ~2 seconds to <0.1 seconds.

11.3.4 Configuration (`config.json`)

The `config.json` file decouples hard-coded values from the codebase. It likely contains:

- **Similarity Thresholds:** Values (e.g., 0.6 or 0.75) determining how close a match must be to be accepted.
- **API Keys:** Credentials for external services like the DuckDuckGo or Wikipedia APIs.
- **Debug Mode:** Flags to enable or disable verbose logging during development.

12. Algorithm Description

The following pseudocode describes the operational logic of the `get_response` function:

Plaintext

```
ALGORITHM GetResponse(userInput):
    1. Load JSON dataset into memory (knowledgeBase)
    2. cleanInput = Preprocess(userInput) -> lowercase, remove punctuation
    3. Initialize maxScore = 0
    4. Initialize bestAnswer = null

    5. FOR EACH entry IN knowledgeBase:
        a. cleanQuestion = Preprocess(entry.question)
        b. currentScore = CalculateSimilarity(cleanInput, cleanQuestion)
        c. IF currentScore > maxScore:
            i. maxScore = currentScore
            ii. bestAnswer = entry.answer

    6. IF maxScore > CONFIDENCE_THRESHOLD (e.g., 0.6):
        RETURN bestAnswer
    7. ELSE:
        wikiResult = SearchWikipedia(userInput)
        IF wikiResult IS NOT NULL:
            RETURN wikiResult
        ELSE:
            RETURN "I am sorry, I do not understand the question."
```

13. Testing Strategy

13.1 User Acceptance Testing (UAT)

A small group of 5 students was asked to use the chatbot. They were given a list of topics but no specific questions, to test how the bot handles natural variation in phrasing.

Sample Test Cases:

Query	Expected Outcome	Actual Outcome	Status
"What is AI?"	Database Definition	Database Definition	Pass
"Tell me about AI"	Database Definition	Database Definition	Pass
"Who is Elon Musk?"	Wikipedia Bio	Wikipedia Bio	Pass
"sdlfkjsd"	Error Message	Error Message	Pass

14. Results and Discussion

14.1 Performance Analysis

The chatbot demonstrated a high success rate for questions that shared significant keywords with the database entries.

- **Local Lookup Speed:** < 0.1 seconds.
- **API Lookup Speed:** $1.5 - 2.0$ seconds (dependent on internet speed).
- **Accuracy:** Approximately 85% for in-domain questions (e.g., Python syntax questions provided in the dataset).

14.2 Impact of Preprocessing

Removing stop words significantly improved accuracy. Without stop-word removal, the query "What is the Python?" might match poorly against "What is Python" if the algorithm weighted "the" too heavily.

14.3 Web Fallback Efficacy

The integration of Wikipedia proved crucial. During testing, users asked about historical figures and countries (data not in JSON). The system seamlessly fetched this data, creating the illusion of a much larger knowledge base.

15. Demonstration

The final product is a web page titled "AI FAQ Assistant".

- **Visuals:** Clean white background with a central chat container.
- **Interaction:**
    1. User types: "How do I install Flask?"
    2. Chatbot displays: "Typing..." animation.
    3. Chatbot replies: "You can install Flask using pip. Run the command `pip install Flask` in your terminal."

16. Limitations

While the project met its objectives, several limitations adhere to its simplified design:

1. **Lack of Semantic Understanding:** The bot uses keyword overlap. It does not truly "understand" meaning. For example, "How do I fix a broken loop?" might not match "Debugging infinite loops" if the words are too different, even though the concept is the same.
2. **Context Amnesia:** The bot cannot handle follow-up questions. If a user asks "Who is the president of the US?" and then asks "How old is **he**?", the bot will not know who "he" refers to.

3. **Spelling Sensitivity:** The basic similarity algorithm is sensitive to typos. "Pytthon" (with two t's) might fail to match "Python".

17. Security and Ethical Considerations

17.1 Data Privacy

The system is designed with privacy in mind.

- **No Persistence:** User queries are processed in RAM and not saved to a persistent database.
- **No PII:** The system does not ask for or store Personally Identifiable Information (email, names, etc.).

17.2 Bias in Public Data

Since the chatbot retrieves data from Wikipedia and user-generated FAQs, there is a risk of propagating biased information found in those sources. However, as a retrieval-based system, it does not *hallucinate* false facts like generative AI, it only repeats what is in the source.

18. Future Enhancements

To upgrade this academic project to a production-ready system, the following enhancements are proposed:

1. **Machine Learning Classifier:** Replace simple similarity matching with a trained classifier (e.g., Naive Bayes or Support Vector Machine) to categorize user intent with higher accuracy.
2. **Deep Learning Integration:** Implement a BERT (Bidirectional Encoder Representations from Transformers) model to generate embeddings. This would solve the "semantic understanding" limitation.
3. **Voice Interaction:** Integrate the Web Speech API to allow users to speak their questions.
4. **Feedback Loop:** Add a "Was this helpful? Yes/No" button. If the user clicks "No", the system logs the query for manual review to improve the dataset.

20. Conclusion

The AI FAQ Chatbot project successfully demonstrates the viability of using introductory AI and NLP techniques to solve real-world information retrieval problems. By combining a local knowledge base with the vast resources of the internet (via APIs), the system achieves a balance between speed, accuracy, and breadth of knowledge.

While it lacks the conversational nuance of advanced Large Language Models, its transparency, low resource usage, and determinism make it an excellent model for specific domain applications (like a university help desk) and a valuable educational tool for understanding the mechanics of Artificial Intelligence.

21. References

1. **Bird, S., Klein, E., & Loper, E.** (2009). *Natural Language Processing with Python*. O'Reilly Media.
2. **Flask Documentation.** (2023). *User's Guide*. Retrieved from flask.palletsprojects.com
3. **Wikipedia API.** (2023). *MediaWiki Action API Documentation*. Retrieved from mediawiki.org
4. **Jurafsky, D., & Martin, J. H.** (2021). *Speech and Language Processing*. Stanford University.
5. **Python Software Foundation.** (2023). *Python 3.11.4 Documentation*. Retrieved from docs.python.org